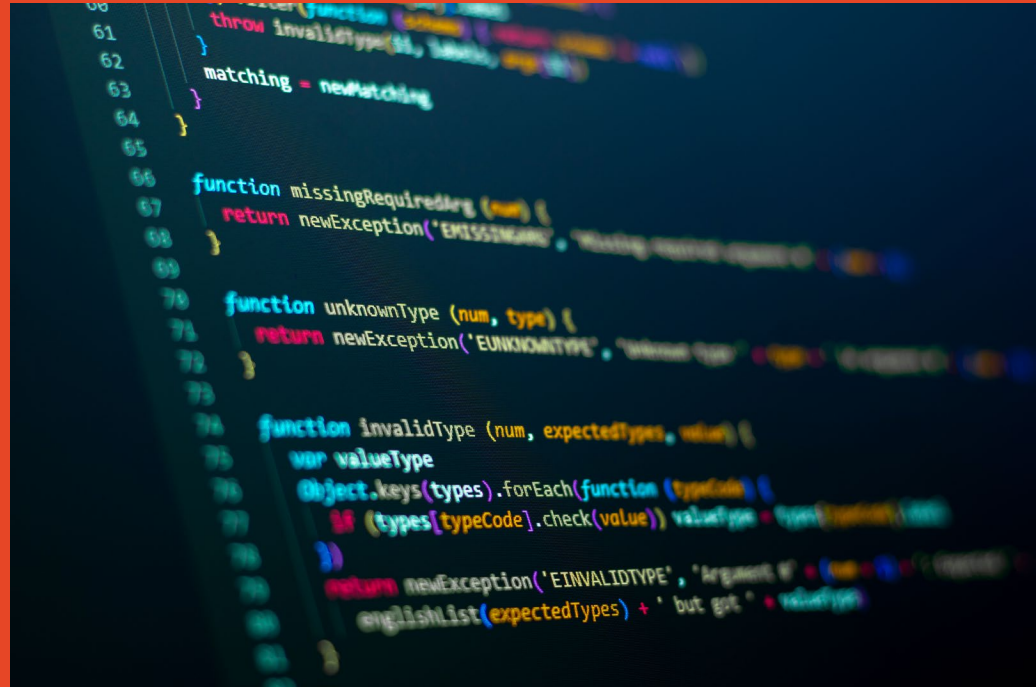


Week 4

Operators, Conditionals and Loops



THE UNIVERSITY OF
SYDNEY



This week

- We will learn two foundational concepts of JavaScript.
- Boolean Operators
- Conditionals
- Loops

Boolean Operators Introduction

- We covered during week 2 different types of math operations.
- There is another type, which is also a type of “math” called Boolean operators.
- Boolean values can be *True* or *False*.
- Boolean Operators combine *True* and *False* to ask more complex yes/no questions.
- Let's see more.

Comparison Operators

Test whether the expression is **TRUE** or **FALSE**.

// Assignment

let x = 1;

// Comparison Expressions

x > 1

Comparison Operators

// Assignment

let x = 1;

// Comparison Expressions

x > 1 // False

Comparison Operators

// Assignment

let x = 1;

// Comparison Expressions

x > 1 // False

x >= 1

Comparison Operators

// Assignment

let *x* = 1;

// Comparison Expressions

x > 1 *// False*

x >= 1 *// True*

Comparison Operators

// Assignment

let x = 1;

// Comparison Expressions

x > 1 // False

x >= 1 // True

x < 1

Comparison Operators

// Assignment

let x = 1;

// Comparison Expressions

x > 1 // False

x >= 1 // True

x < 1 // False

Comparison Operators

// Assignment

let x = 1;

// Comparison Expressions

x > 1 // False

x >= 1 // True

x < 1 // False

x <= 1

Comparison Operators

// Assignment

let x = 1;

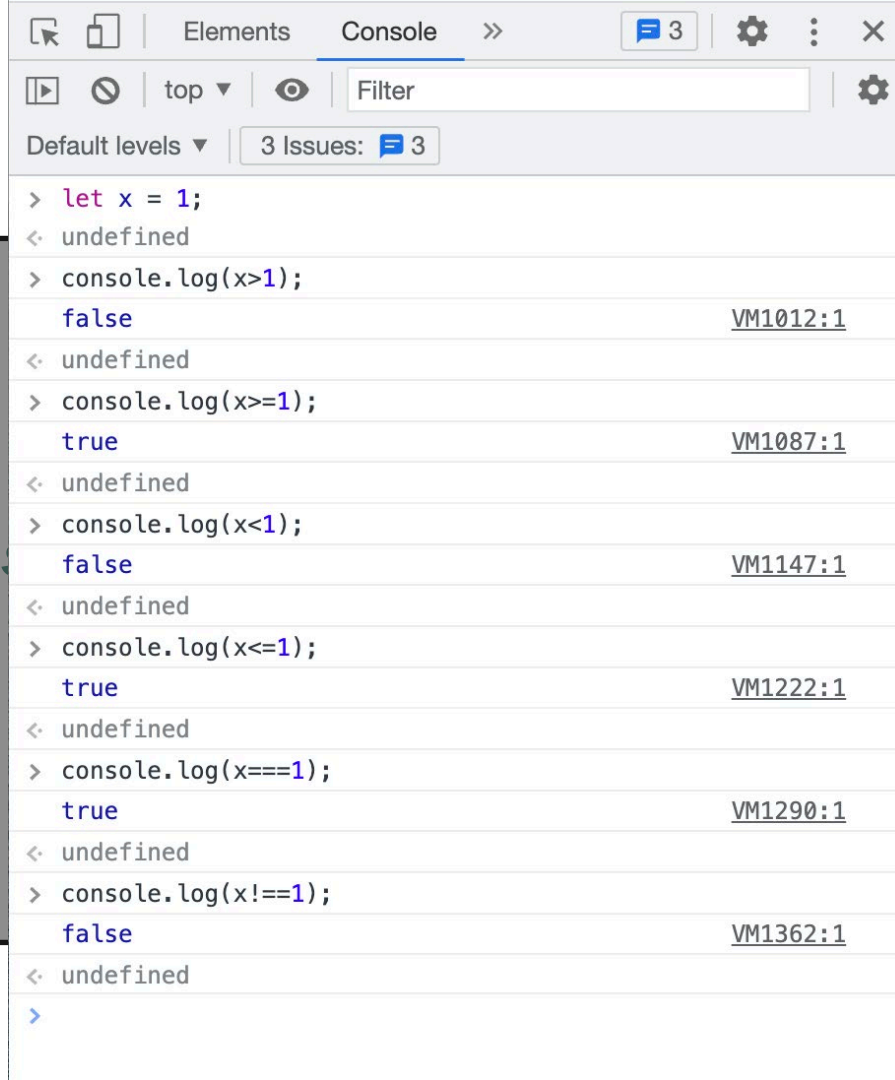
// Comparison Expressions

x > 1 // False

x >= 1 // True

x < 1 // False

x <= 1 // True



The screenshot shows a web browser's developer console with the 'Console' tab selected. The console displays the results of several JavaScript commands entered in the prompt. The commands and their outputs are as follows:

Command	Output	Source
> let x = 1;	undefined	
> console.log(x>1);	false	VM1012:1
< undefined		
> console.log(x>=1);	true	VM1087:1
< undefined		
> console.log(x<1);	false	VM1147:1
< undefined		
> console.log(x<=1);	true	VM1222:1
< undefined		
> console.log(x===1);	true	VM1290:1
< undefined		
> console.log(x!==1);	false	VM1362:1
< undefined		
>		

Comparison Operators

- A comparison with two `==` signs are called LOOSE EQUALITY.
- It checks only for values.

// Comparison Expressions

1 == 1 // True

'1' == 1 // True – interpreted as 1 == 1

true == 1 // True – interpreted as 1 == 1

Comparison Operators

- A comparison with two `==` signs are called LOOSE EQUALITY.
- It checks only for values.

```
// Comparison Expressions
```

```
1 == 1 // True
```

```
'1' == 1 // True – interpreted as 1 == 1
```

```
true == 1 // True – interpreted as 1 == 1
```

JavaScript always tries to convert both to numbers

Comparison Operators

- A comparison with three **===** signs is called STRICT EQUALITY.
- It checks for types and values.

// Comparison Expressions

1 === 1 // True

'1' === '1' // True

'1' === 1 // False

true === 1 // False

Nullish Coalescing Operator

```
/**  
 * If the user specifies a colour,  
 * set that colour, else,  
 * set the default colour to blue.  
 **/  
let userChoice = 'red';  
let color = userChoice ?? 'blue';
```

Nullish Coalescing Operator

```
/**  
 * If the user specifies a colour,  
 * set that colour, else,  
 * set the default colour to blue.  
 **/  
let userChoice = 'red';  
let color = userChoice ?? 'blue';
```

Check if userChoice has a value.

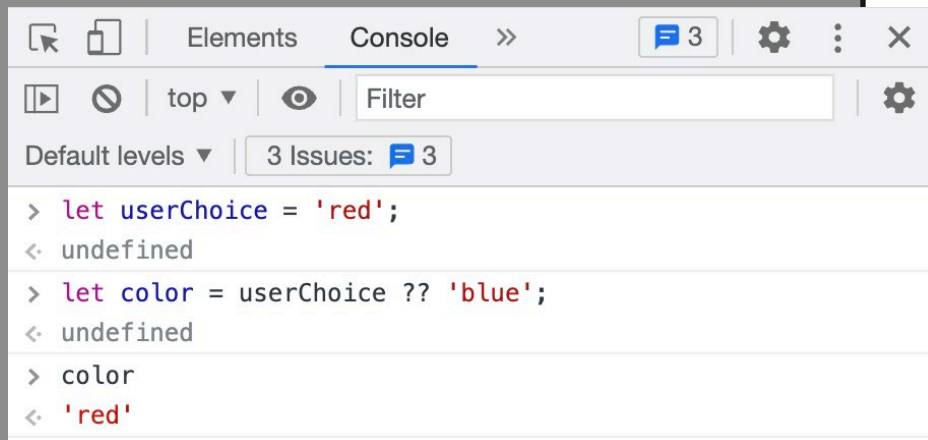
Nullish Coalescing Operator

```
/**  
 * If the user specifies a colour,  
 * set that colour, else,  
 * set the default colour to blue.  
 **/  
let userChoice = 'red';  
let color = userChoice ?? 'blue';
```

userChoice has the value of red,
which means that color will be red.

Nullish Coalescing Operator

```
/**  
 * If the user specifies a colour,  
 * set that colour, else,  
 * set the default colour to blue.  
 */  
let userChoice = 'red';  
let color = userChoice ?? 'blue';
```



Nullish Coalescing Operator

```
/**  
 * If the user specifies a colour,  
 * set that colour, else,  
 * set the default colour to blue.  
 **/  
let userChoice = null;  
let color = userChoice ?? 'blue';
```

Nullish Coalescing Operator

```
/**  
 * If the user specifies a colour,  
 * set that colour, else,  
 * set the default colour to blue.  
 **/  
let userChoice = null;  
let color = userChoice ?? 'blue';
```

userChoice has the value of null.

Nullish Coalescing Operator

```
/**  
* If the user specifies a colour,  
* set that colour, else,  
* set the default colour to blue.  
**/  
let userChoice = null;  
let color = userChoice ?? 'blue';
```

userChoice has the value of null,
color will be blue

Nullish Coalescing Operator

```
/**
```

```
* If the user specifies a colour,
```

```
* set that colour, else,
```

```
* set the default colour to blue.
```

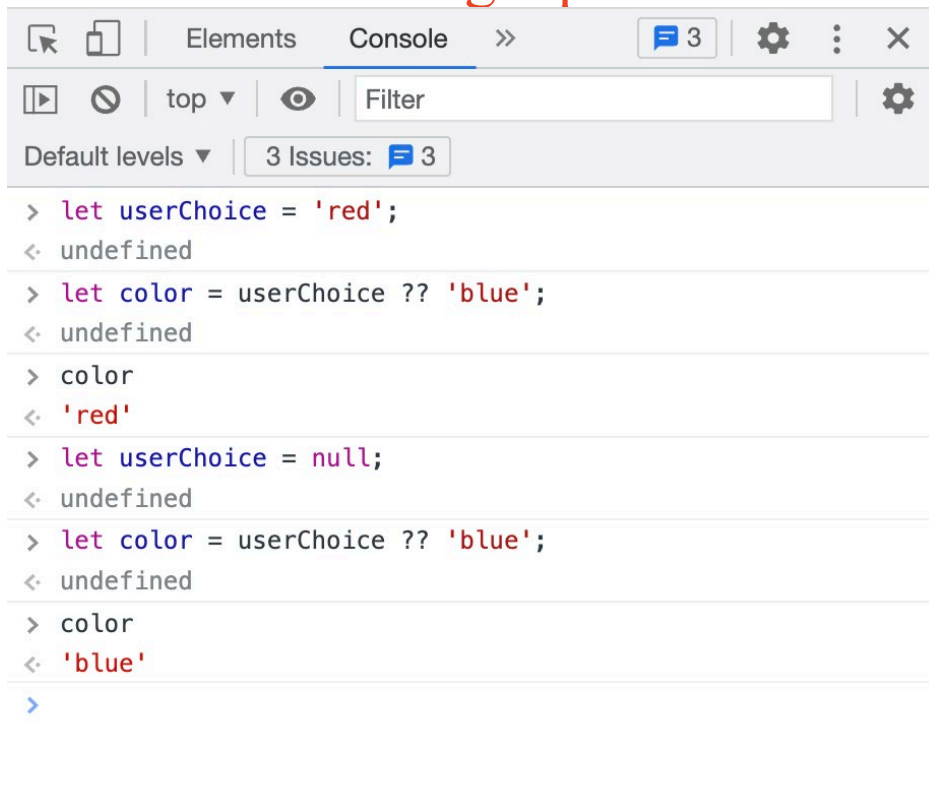
```
*/
```

```
let userChoice = null;
```

```
let color = userChoice ?? 'blue';
```

```
> let userChoice = null;
< undefined
> let color = userChoice ?? 'blue';
< undefined
> color
< 'blue'
>
```

Nullish Coalescing Operator



```
> let userChoice = 'red';  
< undefined  
  
> let color = userChoice ?? 'blue';  
< undefined  
  
> color  
< 'red'  
  
> let userChoice = null;  
< undefined  
  
> let color = userChoice ?? 'blue';  
< undefined  
  
> color  
< 'blue'  
  
>
```

We know it is a tricky expression, so be ready to ask all about it in your tutorials 😊

Logical Operators

- There are three basic logic operators:
 - AND (&&)
 - OR (||)
 - NOT (!)

Logical Operators (AND)

Input 1	Input 2	Output
True(1)	True (1)	True(1)
True(1)	False (0)	False(0)
False (0)	True (1)	False (0)
False (0)	False (0)	False (0)

Logical Operators

Input 1	Input 2	Output
True(1)	True (1)	True(1)
True(1)	False (0)	False(0)
False (0)	True (1)	False (0)
False (0)	False (0)	False (0)

// AND - &&
// Returns true if BOTH are True.

True && True;

Logical Operators

Input 1	Input 2	Output
True(1)	True (1)	True(1)
True(1)	False (0)	False(0)
False (0)	True (1)	False (0)
False (0)	False (0)	False (0)

***// AND - &&
// Returns true if BOTH are True.***

True && True; // returns True

Logical Operators

Input 1	Input 2	Output
True(1)	True (1)	True(1)
True(1)	False (0)	False(0)
False (0)	True (1)	False (0)
False (0)	False (0)	False (0)

***// AND - &&
// Returns true if BOTH are True.***

***True && True; // returns True
True && False;***

Logical Operators

Input 1	Input 2	Output
True(1)	True (1)	True(1)
True(1)	False (0)	False(0)
False (0)	True (1)	False (0)
False (0)	False (0)	False (0)

// AND - &&
// Returns true if BOTH are True.

True && True; // returns True
True && False; // returns False

Logical Operators

Input 1	Input 2	Output
True(1)	True (1)	True(1)
True(1)	False (0)	False(0)
False (0)	True (1)	False (0)
False (0)	False (0)	False (0)

// AND - &&

// Returns true if BOTH are True.

True && True; // returns True

True && False; // returns False

3 < 5 && 1 < 2;

Logical Operators

Input 1	Input 2	Output
True(1)	True (1)	True(1)
True(1)	False (0)	False(0)
False (0)	True (1)	False (0)
False (0)	False (0)	False (0)

// AND - &&

// Returns true if BOTH are True.

True && True; // returns True

True && False; // returns False

3 < 5 && 1 < 2; // returns True

Logical Operators

Input 1	Input 2	Output
True(1)	True (1)	True(1)
True(1)	False (0)	False(0)
False (0)	True (1)	False (0)
False (0)	False (0)	False (0)

// AND - &&

// Returns true if BOTH are True.

True && True; // returns True

True && False; // returns False

3 < 5 && 1 < 2; // returns True

// True && True; return True

Logical Operators (OR)

Input 1	Input 2	Output
True(1)	True (1)	True(1)
True(1)	False (0)	True(1)
False (0)	True (1)	True(1)
False (0)	False (0)	False (0)

Logical Operators

Input 1	Input 2	Output
True(1)	True (1)	True(1)
True(1)	False (0)	True(1)
False (0)	True (1)	True(1)
False (0)	False (0)	False (0)

// OR - ||
// Returns true if ANY is True.

True || True;

Logical Operators

Input 1	Input 2	Output
True(1)	True (1)	True(1)
True(1)	False (0)	True(1)
False (0)	True (1)	True(1)
False (0)	False (0)	False (0)

// OR - ||
// Returns true if ANY is True.

True || True; // returns True

Logical Operators

Input 1	Input 2	Output
True(1)	True (1)	True(1)
True(1)	False (0)	True(1)
False (0)	True (1)	True(1)
False (0)	False (0)	False (0)

// OR - ||

// Returns true if ANY is True.

True || True; // returns True

False || False;

Logical Operators

Input 1	Input 2	Output
True(1)	True (1)	True(1)
True(1)	False (0)	True(1)
False (0)	True (1)	True(1)
False (0)	False (0)	False (0)

// OR - ||

// Returns true if ANY is True.

True || True; // returns True

False || False; // returns False

Logical Operators

Input 1	Input 2	Output
True(1)	True (1)	True(1)
True(1)	False (0)	True(1)
False (0)	True (1)	True(1)
False (0)	False (0)	False (0)

// OR - ||

// Returns true if ANY is True.

True || True; // returns True

False || False; // returns False

True || False;

Logical Operators

Input 1	Input 2	Output
True(1)	True (1)	True(1)
True(1)	False (0)	True(1)
False (0)	True (1)	True(1)
False (0)	False (0)	False (0)

// OR - ||

// Returns true if ANY is True.

True || True; // returns True

False || False; // returns False

True || False; // returns True

Logical Operators

Input 1	Input 2	Output
True(1)	True (1)	True(1)
True(1)	False (0)	True(1)
False (0)	True (1)	True(1)
False (0)	False (0)	False (0)

// OR - ||

// Returns true if ANY is True.

True || True; // returns True

False || False; // returns False

True || False; // returns True

3 < 5 || 1 < 2;

Logical Operators

Input 1	Input 2	Output
True(1)	True (1)	True(1)
True(1)	False (0)	True(1)
False (0)	True (1)	True(1)
False (0)	False (0)	False (0)

// OR - ||

// Returns true if ANY is True.

True || True; // returns True

False || False; // returns False

True || False; // returns True

3 < 5 || 1 < 2; // returns True

Logical Operators

Input 1	Input 2	Output
True(1)	True (1)	True(1)
True(1)	False (0)	True(1)
False (0)	True (1)	True(1)
False (0)	False (0)	False (0)

// OR - ||

// Returns true if ANY is True.

True || True; // returns True

False || False; // returns False

True || False; // returns True

3 < 5 || 1 < 2; // returns True

// True || True; return True

Logical Operators (NOT)

Input 1	Output
True(1)	False (0)
False (0)	True (1)

Logical Operators

Input 1	Output
True(1)	False (0)
False (0)	True (1)

// NOT - !

// Returns the opposite.

!true; // returns False

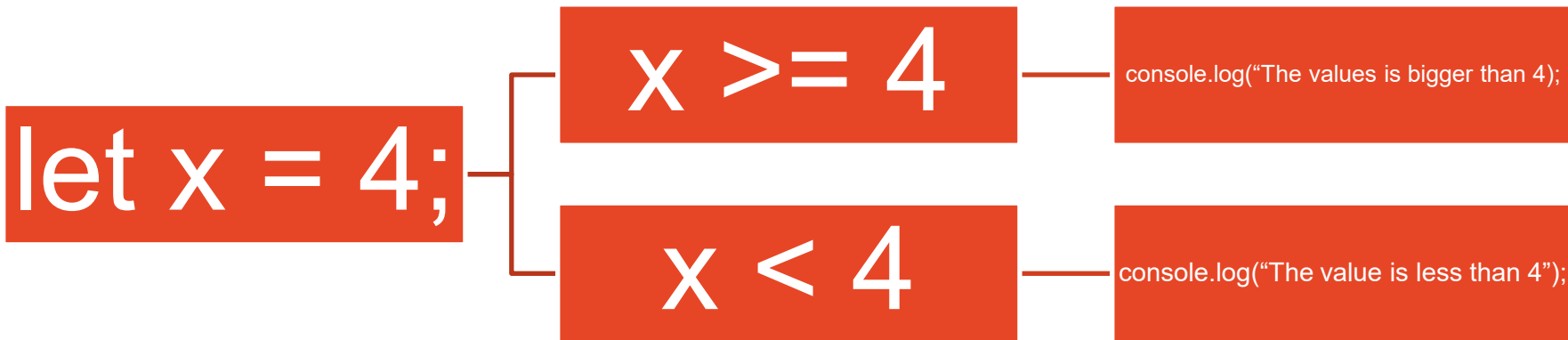
!false; // returns True

! (1 < 2); // returns False

// ! (True) returns False

Conditionals

- Conditionals are one of the simplest ways to introduce interactivity and flow to your programs.
- In the simplest form, conditionals are logical tests that can evaluate to either TRUE or FALSE, and that proceed to execute some code based on the value.



Conditional 1: If

```
// Assign Value
```

```
let x = 4;
```

```
// Check if the value is bigger than 4
```

```
if (x >= 4) {
```

```
  console.log ("The value is bigger than 4");
```

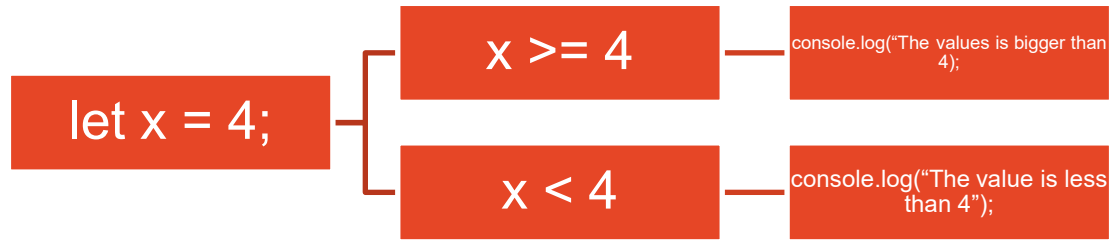
```
}
```

```
if (x < 4) {
```

```
  console.log ("The value is lower than 4");
```

```
}
```

Conditional 1: If



// Assign Value

let x = 4;

// Check if the value is bigger than 4

if (x >= 4) {

console.log ("The value is bigger than 4");

}

if (x < 4) {

console.log ("The value is lower than 4");

}

Conditional 1: If

```
// Assign Value
```

```
let x = 4;
```

```
// Check if the value is bigger than 4
```

```
if (x >= 4) {  
  console.log ("The value is bigger than 4");  
}
```

```
if (x < 4) {  
  console.log ("The value is lower than 4");  
}
```

This is what an If conditional looks like.

Conditional 1: If

```
// Assign Value
```

```
let x = 4;
```

```
// Check if the value is bigger than 4
```

```
if (x >= 4) {
```

```
  console.log ("The value is bigger than 4");  
}
```

```
if (x < 4) {
```

```
  console.log ("The value is lower than 4");  
}
```

The keyword “if” starts the conditional.

After the if, you include the condition, which should always be a Boolean condition (check Week 3!)

Conditional 1: If

```
// Assign Value
```

```
let x = 4;
```

```
// Check if the value is bigger than 4
```

```
if (x >= 4) {
```

```
  console.log ("The value is bigger than 4");
```

```
}
```

```
if (x < 4) {
```

```
  console.log ("The value is lower than 4");
```

```
}
```

- The content is inside the curly brackets.
- It will ONLY execute if the condition returns TRUE.
- Otherwise, it will be ignored.

Conditional 1: If- Else

```
// Assign Value
```

```
let x = 1;
```

```
let color;
```

```
// Check if the value is bigger than 4
```

```
if (x >= 4) {
```

```
    color = "red";
```

```
}
```

```
else {
```

```
    color = "blue";
```

```
}
```

Conditional 1: If- Else

```
// Assign Value
```

```
let x = 1;
```

```
let color;
```

```
// Check if the value is bigger than 4
```

```
if (x >= 4) {
```

```
    color = "red"; // Will color be red?
```

```
}
```

```
else {
```

```
    color = "blue"; // Will color be blue?
```

```
}
```

Conditional 1: If- Else

```
// Assign Value  
let x = 1;  
let color;  
  
// Check if the value is bigger than 4  
if (x >= 4) {  
    color = "red"; // Wrong Answer  
}  
else {  
    color = "blue"; // Right Answer!  
}
```

Conditional 1: If- Else

```
// Assign Value  
let x = 1;  
let color;  
  
// Check if the value is bigger than 4  
if (x >= 4) {  
  color = "red";  
}  
else {  
  color = "blue";  
}
```

Else is a way to say, “You do not care about the value, if it is not the first condition, then do something else”.

Conditional 1: If- Else

```
// Assign Value
```

```
let x = 5;
```

```
let color;
```

```
//
```

```
if (x >= 20) {
```

```
    color = "red";
```

```
}
```

```
else if (x < 20 && x >= 10) {
```

```
    color = "blue";
```

```
}
```

```
else {
```

```
    color = "white";
```

```
}
```

Conditional 1: If- Else

```
// Assign Value
```

```
let x = 5;
```

```
let color;
```

```
//
```

```
if (x >= 20) {
```

```
    color = "red";
```

```
}
```

```
else if (x < 20 && x >= 10) {
```

```
    color = "blue";
```

```
}
```

```
else {
```

```
    color = "white";
```

```
}
```

Is x greater or equal to 20?

False. Check the next

Conditional 1: If- Else

```
// Assign Value
```

```
let x = 5;
```

```
let color;
```

```
//
```

```
if (x >= 20) {
```

```
    color = "red";
```

```
}
```

```
else if (x < 20 && x >= 10) {
```

```
    color = "blue";
```

```
}
```

```
else {
```

```
    color = "white";
```

```
}
```

Is x less than 20?

AND

X bigger or equal to 10?

Remember Week 3.

x < 20 : False

x >= 10: False

False AND False: False

Because if False, check next

Conditional 1: If- Else

```
// Assign Value
```

```
let x = 5;
```

```
let color;
```

```
//
```

```
if (x >= 20) {
```

```
  color = "red";
```

```
}
```

```
else if (x < 20 && x >= 10) {
```

```
  color = "blue";
```

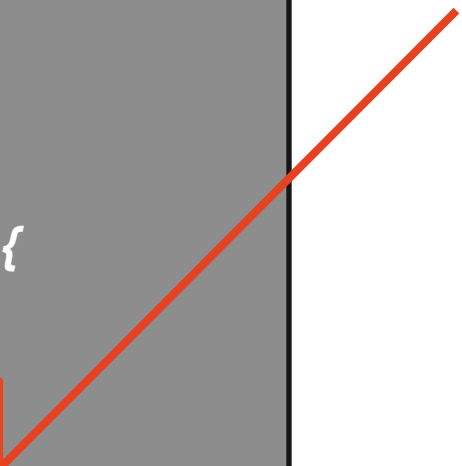
```
}
```

```
else {
```

```
  color = "white";
```

```
}
```

The result is White!



Conditional 1: If – Else closing thoughts

- If and Else statements are building blocks of your code.
- You will see yourself using them increasingly as your code becomes more complex.
- You will see how they work in your tutorial.

Food for thought

- Imagine you calculate the price of an item based on the customer type

```
// Example of a verbose if-else chain to determine discount based on customer type
function calculateDiscount(customerType) {
  // Get the base price from somewhere
  const basePrice = 100;
  let finalPrice;
  let discountPercentage;

  if (customerType === 'new') {
    discountPercentage = 5;
    console.log('New customer discount applied: 5%');
    finalPrice = basePrice - (basePrice * 0.05);
  } else if (customerType === 'regular') {
    discountPercentage = 10;
    console.log('Regular customer discount applied: 10%');
    finalPrice = basePrice - (basePrice * 0.1);
  } else if (customerType === 'premium') {
    discountPercentage = 15;
    console.log('Premium customer discount applied: 15%');
    finalPrice = basePrice - (basePrice * 0.15);
  } else if (customerType === 'gold') {
    discountPercentage = 20;
    console.log('Gold customer discount applied: 20%');
    finalPrice = basePrice - (basePrice * 0.2);
  } else if (customerType === 'platinum') {
    discountPercentage = 25;
    console.log('Platinum customer discount applied: 25%');
    finalPrice = basePrice - (basePrice * 0.25);
  } else if (customerType === 'diamond') {
    discountPercentage = 30;
    console.log('Diamond customer discount applied: 30%');
    finalPrice = basePrice - (basePrice * 0.3);
  } else if (customerType === 'vip') {
    discountPercentage = 35;
    console.log('VIP customer discount applied: 35%');
    finalPrice = basePrice - (basePrice * 0.35);
  } else if (customerType === 'family') {
    discountPercentage = 40;
    console.log('Family customer discount applied: 40%');
    finalPrice = basePrice - (basePrice * 0.4);
  } else if (customerType === 'employee') {
    discountPercentage = 50;
    console.log('Employee discount applied: 50%');
  }
}
```

price of an item
;

Conditional 2:

- Serves the exact same purpose as the if statement
- Useful when you have multiple conditions to check
- So, how does this work?

```
// The same logic implemented with a switch statement
function calculateDiscountWithSwitch(customerType) {
  const basePrice = 100;
  let finalPrice;
  let discountPercentage;

  switch (customerType) {
    case 'new':
      discountPercentage = 5;
      console.log('New customer discount applied: 5%');
      finalPrice = basePrice - (basePrice * 0.05);
      break;
    case 'regular':
      discountPercentage = 10;
      console.log('Regular customer discount applied: 10%');
      finalPrice = basePrice - (basePrice * 0.1);
      break;
    case 'premium':
      discountPercentage = 15;
      console.log('Premium customer discount applied: 15%');
      finalPrice = basePrice - (basePrice * 0.15);
      break;
    case 'gold':
      discountPercentage = 20;
      console.log('Gold customer discount applied: 20%');
      finalPrice = basePrice - (basePrice * 0.2);
      break;
    case 'platinum':
      discountPercentage = 25;
      console.log('Platinum customer discount applied: 25%');
      finalPrice = basePrice - (basePrice * 0.25);
      break;
    case 'diamond':
      discountPercentage = 30;
      console.log('Diamond customer discount applied: 30%');
      finalPrice = basePrice - (basePrice * 0.3);
      break;
    case 'vip':
      discountPercentage = 35;
      console.log('VIP customer discount applied: 35%');
      finalPrice = basePrice - (basePrice * 0.35);
      break;
  }
}
```

```
// Example of a verbose if-else chain to determine discount based on customer type
function calculateDiscount(customerType) {
  // Get the base price from somewhere
  const basePrice = 100;
  let finalPrice;
  let discountPercentage;

  if (customerType === 'new') {
    discountPercentage = 5;
    console.log('New customer discount applied: 5%');
    finalPrice = basePrice - (basePrice * 0.05);
  } else if (customerType === 'regular') {
    discountPercentage = 10;
    console.log('Regular customer discount applied: 10%');
    finalPrice = basePrice - (basePrice * 0.1);
  } else if (customerType === 'premium') {
    discountPercentage = 15;
    console.log('Premium customer discount applied: 15%');
    finalPrice = basePrice - (basePrice * 0.15);
  } else if (customerType === 'gold') {
    discountPercentage = 20;
    console.log('Gold customer discount applied: 20%');
    finalPrice = basePrice - (basePrice * 0.2);
  } else if (customerType === 'platinum') {
    discountPercentage = 25;
    console.log('Platinum customer discount applied: 25%');
    finalPrice = basePrice - (basePrice * 0.25);
  } else if (customerType === 'diamond') {
    discountPercentage = 30;
    console.log('Diamond customer discount applied: 30%');
    finalPrice = basePrice - (basePrice * 0.3);
  } else if (customerType === 'vip') {
    discountPercentage = 35;
    console.log('VIP customer discount applied: 35%');
    finalPrice = basePrice - (basePrice * 0.35);
  } else if (customerType === 'family') {
    discountPercentage = 40;
    console.log('Family customer discount applied: 40%');
    finalPrice = basePrice - (basePrice * 0.4);
  } else if (customerType === 'employee') {
    discountPercentage = 50;
    console.log('Employee discount applied: 50%');
```

```
// The same logic implemented with a switch statement
function calculateDiscountWithSwitch(customerType) {
  const basePrice = 100;
  let finalPrice;
  let discountPercentage;

  switch (customerType) {
    case 'new':
      discountPercentage = 5;
      console.log('New customer discount applied: 5%');
      finalPrice = basePrice - (basePrice * 0.05);
      break;
    case 'regular':
      discountPercentage = 10;
      console.log('Regular customer discount applied: 10%');
      finalPrice = basePrice - (basePrice * 0.1);
      break;
    case 'premium':
      discountPercentage = 15;
      console.log('Premium customer discount applied: 15%');
      finalPrice = basePrice - (basePrice * 0.15);
      break;
    case 'gold':
      discountPercentage = 20;
      console.log('Gold customer discount applied: 20%');
      finalPrice = basePrice - (basePrice * 0.2);
      break;
    case 'platinum':
      discountPercentage = 25;
      console.log('Platinum customer discount applied: 25%');
      finalPrice = basePrice - (basePrice * 0.25);
      break;
    case 'diamond':
      discountPercentage = 30;
      console.log('Diamond customer discount applied: 30%');
      finalPrice = basePrice - (basePrice * 0.3);
      break;
    case 'vip':
      discountPercentage = 35;
      console.log('VIP customer discount applied: 35%');
      finalPrice = basePrice - (basePrice * 0.35);
      break;
```

```
let x = 2;  
let color;
```

```
switch (x) {
```

```
  case 1:
```

```
    color = "red";
```

```
    break;
```

```
  case 2:
```

```
    color = "blue";
```

```
    break;
```

```
  case 3:
```

```
    color = "white";
```

```
    break;
```

```
  default:
```

```
    color = "black";
```

```
    break;
```

```
}
```

Conditional 2: Switch-Case

Keyword switch and the variable we compare.

Conditional 2: Switch-Case

```
let x = 2;  
let color;  
  
switch (x) {  
  case 1:  
    color = "red";  
    break;  
  case 2:  
    color = "blue";  
    break;  
  case 3:  
    color = "white";  
    break;  
  default:  
    color = "black";  
    break;  
}
```

Case keyword means:
“in case the value is 1”

Conditional 2: Switch-Case

```
let x = 2;  
let color;  
  
switch (x) {  
  case 1:  
    color = "red";  
    break;  
  case 2:  
    color = "blue";  
    break;  
  case 3:  
    color = "white";  
    break;  
  default:  
    color = "black";  
    break;  
}
```

Break means:
“found the result, skip the
rest.”

Conditional 2: Switch-Case

```
let x = 2;  
let color;  
  
switch (x) {  
  case 1:  
    color = "red";  
    break;  
  case 2:  
    color = "blue";  
    break;  
  case 3:  
    color = "white";  
    break;  
  default:  
    color = "black";  
    break;  
}
```

Default keyword means:
"If the value is none of these
options, use the option here"

When to use if-else and when to use switch-case?

- It is all about personal preference.
- If you have complex comparisons like (**$x > 50 \ \&\& \ y < 30 \ \&\& \ z == 23$**) use if.
- If you have a lot of potential values (like the discount example), use switch.
- Otherwise, use whatever option you think works better 😊

Imagine

```
function someFunction(inputValue){  
    console.log(`The input value is ${inputValue}`)  
}
```

```
function printRandomNumbers(){  
    someFunction(Math.random() * 100);  
    someFunction(Math.random() * 100);  
    someFunction(Math.random() * 100);  
    someFunction(Math.random() * 100);  
    someFunction(Math.random() * 100);  
    someFunction(Math.random() * 100);  
    someFunction(Math.random() * 100);  
    someFunction(Math.random() * 100);  
    someFunction(Math.random() * 100);  
    someFunction(Math.random() * 100);  
}
```

For Loops

- For loops, allow us to make repetitive tasks clean and efficiently.
- It allows you to repeat a block of code multiple times.

```
function printRandomNumbersBetter(){  
  for (let i = 0; i < 10; i++){  
    someFunction(Math.random() * 100);  
  }  
}
```

For Loops

```
function printRandomNumbersBetter(){  
  for (let i = 0; i < 10; i++){  
    someFunction(Math.random() * 100);  
  }  
}
```

```
function printRandomNumbers(){  
  someFunction(Math.random() * 100);  
  someFunction(Math.random() * 100);  
  someFunction(Math.random() * 100);  
  someFunction(Math.random() * 100);  
  someFunction(Math.random() * 100);  
  someFunction(Math.random() * 100);  
  someFunction(Math.random() * 100);  
  someFunction(Math.random() * 100);  
  someFunction(Math.random() * 100);  
  someFunction(Math.random() * 100);  
}
```

For Loops

Declare and initialise a loop control variable. In this case we want to start at zero.

Set the loop continuation condition. In this case it is anything less than the value of maximum size.

Set the increment. This is what happens to the loop control variable every iteration.

```
for (let i = 0; i < maximumSize; i++) {  
  console.log(i);  
}
```

This is the body of the loop, it is run every iteration.

Let's look at this

```
for (let i = 0; i < 3; i++){  
  console.log(`loop: ${i + 1}`);  
}
```

Let's look at this

```
for (let i = 0; i < 3; i++){
```

```
  console.log(`loop: ${i + 1}`);
```

```
}
```

Program status:
1) $i = 0$

Let's look at this

```
for (let i = 0; i < 3; i++){
```

```
  console.log(`loop: ${i + 1}`);
```

```
}
```

Program status:

- 1) $i = 0$
- 2) $i < 3$? *True*

Let's look at this

```
for (let i = 0; i < 3; i++){
```

```
  console.log(`loop: ${i + 1}`);
```

```
}
```

Program status:

- 1) $i = 0$
- 2) $i < 3$? *True*
- 3) *Print to console: 1*

Let's look at this

```
for (let i = 0; i < 3; i++) {  
  console.log(`loop: ${i + 1}`);  
}
```

Program status:

- 1) $i = 0$
- 2) $i < 3$? *True*
- 3) *Print to console: 1*
- 4) *Increase i by 1*
- 5) $i = 1$

Let's look at this

```
for (let i = 0; i < 3; i++){
```

```
  console.log(`loop: ${i + 1}`),
```

```
}
```

Program status:

- 1) $i = 0$
- 2) $i < 3$? *True*
- 3) *Print to console: 1*
- 4) *Increase i by 1*
- 5) $i = 1$
- 6) $i < 3$? *True*

Let's look at this

```
for (let i = 0; i < 3; i++){
```

```
  console.log(`loop: ${i + 1}`);
```

```
}
```

Program status:

- 1) $i = 0$
- 2) $i < 3$? *True*
- 3) *Print to console: 1*
- 4) *Increase i by 1*
- 5) $i = 1$
- 6) $i < 3$? *True*
- 7) *Print to console: 2*

Let's look at this

```
for (let i = 0; i < 3; i++) {  
  console.log(`loop: ${i + 1}`);  
}
```

Program status:

- 1) $i = 0$
- 2) $i < 3$? True
- 3) Print to console: 1
- 4) Increase i by 1
- 5) $i = 1$
- 6) $i < 3$? True
- 7) Print to console: 2
- 8) Increase i by 1
- 9) $i = 2$

Let's look at this

```
for (let i = 0; i < 3; i++){  
  console.log(`loop: ${i + 1}`);  
}
```

Program status:

- 1) $i = 0$
- 2) $i < 3$? True
- 3) Print to console: 1
- 4) Increase i by 1
- 5) $i = 1$
- 6) $i < 3$? True
- 7) Print to console: 2
- 8) Increase i by 1
- 9) $i = 2$
- 10) $i < 3$? True

Let's look at this

```
for (let i = 0; i < 3; i++){
```

```
  console.log(`loop: ${i + 1}`);
```

```
}
```

Program status:

- 1) $i = 0$
- 2) $i < 3$? *True*
- 3) *Print to console: 1*
- 4) *Increase i by 1*
- 5) $i = 1$
- 6) $i < 3$? *True*
- 7) *Print to console: 2*
- 8) *Increase i by 1*
- 9) $i = 2$
- 10) $i < 3$? *True*
- 11) *Print to console: 3*

Let's look at this

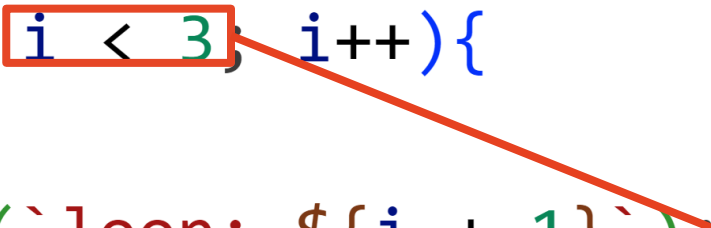
```
for (let i = 0; i < 3; i++) {  
  console.log(`loop: ${i + 1}`);  
}
```

Program status:

- 1) $i = 0$
- 2) $i < 3$? True
- 3) Print to console: 1
- 4) Increase i by 1
- 5) $i = 1$
- 6) $i < 3$? True
- 7) Print to console: 2
- 8) Increase i by 1
- 9) $i = 2$
- 10) $i < 3$? True
- 11) Print to console: 3
- 12) Increase i by 1
- 13) $i = 3$

Let's look at this

```
for (let i = 0; i < 3; i++){  
  console.log(`loop: ${i + 1}`);  
}
```



Program status:

- 1) $i = 0$
- 2) $i < 3?$ True
- 3) Print to console: 1
- 4) Increase i by 1
- 5) $i = 1$
- 6) $i < 3?$ True
- 7) Print to console: 2
- 8) Increase i by 1
- 9) $i = 2$
- 10) $i < 3?$ True
- 11) Print to console: 3
- 12) Increase i by 1
- 13) $i = 3$
- 14) $i < 3?$ False

Let's look at this

```
for (let i = 0; i < 3; i++){  
    console.log(`loop: ${i + 1}`);  
}
```

Program status:

- 1) $i = 0$
- 2) $i < 3?$ True
- 3) Print to console: 1
- 4) Increase i by 1
- 5) $i = 1$
- 6) $i < 3?$ True
- 7) Print to console: 2
- 8) Increase i by 1
- 9) $i = 2$
- 10) $i < 3?$ True
- 11) Print to console: 3
- 12) Increase i by 1
- 13) $i = 3$
- 14) $i < 3?$ False
- 15) Execution Ends

Wrap up everything we learned so far?

Do we want to check all the elements of an array and print whether the number is odd or even?

Wrap up everything we learned so far?

```
let x = [1, 4, 9, 16, 25];
```

```
for (let i = 0; i < x.length; i++){  
  if (x[i] % 2 == 1){  
    console.log(`number ${x[i]} is odd`);  
  } else {  
    console.log(`number ${x[i]} is even`);  
  }  
}
```

Wrap up everything we learned so far?

Array Creation

```
let x = [1, 4, 9, 16, 25];
```

```
for (let i = 0; i < x.length; i++){  
    if (x[i] % 2 == 1){  
        console.log(`number ${x[i]} is odd`);  
    } else {  
        console.log(`number ${x[i]} is even`);  
    }  
}
```

Wrap up everything we learned so far?

```
let x = [1, 4, 9, 16, 25];
```

```
for (let i = 0; i < x.length; i++){
```

For loop

```
    if (x[i] % 2 == 1){
```

```
        console.log(`number ${x[i]} is odd`);
```

```
    } else {
```

```
        console.log(`number ${x[i]} is even`);
```

```
    }
```

```
}
```

Wrap up everything we learned so far?

```
let x = [1, 4, 9, 16, 25];
```

Access an Object's property.

```
for (let i = 0; i < x.length; i++){  
  if (x[i] % 2 == 1){  
    console.log(`number ${x[i]} is odd`);  
  } else {  
    console.log(`number ${x[i]} is even`);  
  }  
}
```

Wrap up everything we learned so far?

```
let x = [1, 4, 9, 16, 25];
```

```
for (let i = 0; i < x.length; i++){
```

```
    if (x[i] % 2 == 1){  
        console.log(`number ${x[i]} is odd`);  
    } else {  
        console.log(`number ${x[i]} is even`);  
    }  
}
```

If – else block



```
}
```

Wrap up everything we learned so far?

```
let x = [1, 4, 9, 16, 25];
```

Use of indexes in an array

```
for (let i = 0; i < x.length; i++){
```

```
    if (x[i] % 2 == 1){
```

```
        console.log(`number ${x[i]} is odd`);
```

```
    } else {
```

```
        console.log(`number ${x[i]} is even`);
```

```
    }
```

```
}
```

Wrap up everything we learned so far?

Try to have a look at this code in your own time.

```
let x = [1, 4, 9, 16, 25];

for (let i = 0; i < x.length; i++){
    if (x[i] % 2 == 1){
        console.log(`number ${x[i]} is odd`);
    } else {
        console.log(`number ${x[i]} is even`);
    }
}
```